```
detachedFromObject = noumenon.itself

detachedFromObject()
# 'undefined'
```

The value of `this` is different. It's `undefined`! The same will happen if you assign the function to the property of another object and make it a method of *that* object:

```
other =
  value: 'other'

other.itself = noumenon.itself

other.itself().value
# 'other'
```

Remember that functions are first-class values? This is the price you have to pay for all that expressive power with dynamic object binding. There's no such thing as a free lunch.

In CoffeeScript, a function never belongs to an object in the same way that methods belong to objects in other object-oriented languages. Instead, a function is bound *dynamically* to the receiving object when invoked. If you pass what you think is a method of an object as a callback function, you're passing a bare-naked first-class function.

If it's still not completely clear, you can experiment with a version of this example on the REPL. First, change the `itself` method to `console.log` so you can see what's happening:

```
noumenon.itself = -> console.log @value
```

Now pass the function to `setTimeout`:

```
setTimeout noumenon.itself, 1000
# 'undefined'
```

As before, the function is being invoked with a different receiver. In the case of `setTimout`, because it's a global method, the function will be